Projet facultatif

Démineur

Les règles du jeu dans une classe, la fenêtre fournie

EPHEC — Bachelier en informatique, niveau 2

Année académique 2026-2027

Ce projet

Un démineur jouable, dans une fenêtre JavaFX. Vous n'en écrivez qu'une classe : Grille, qui contient toutes les règles du jeu. La fenêtre, l'application et son lanceur vous sont fournis, ainsi que onze tests JUnit.

C'est un travail en bonus. Il n'y a rien à remettre et rien n'est évalué. Faites-le seul, à votre rythme, si vous voulez voir une application complète se construire autour d'un modèle que vous avez écrit et testé.

Un corrigé est publié avec cet énoncé, dans un document séparé. Il donne une façon d'écrire chaque méthode, et répond aux questions posées en cours de route. Ne l'ouvrez qu'une fois les tests d'une étape au vert, pour comparer, ou si une étape vous résiste depuis un moment : c'est en cherchant que l'on apprend, pas en lisant la réponse.

Ce que vous recevez

Classe	Rôle	À vous ?
`Grille`	le modèle : les mines, les cases, les règles	en partie
`GrilleTest`	onze tests des règles, sans fenêtre	non, fournie
`VueDemineur`	la vue et le contrôleur : un clic devient un appel à Grille	non, fournie
`Demineur`	l'application : hérite d'Application, remplit la fenêtre	non, fournie
`LanceurDemineur`	le point d'entrée : c'est lui qu'on lance	non, fourni

La règle qui compte

Grille n'importe rien de JavaFX. C'est ce qui la rend testable en une seconde, sans ouvrir de fenêtre, et c'est la séparation entre modèle et vue : la vue demande, le modèle décide. Si vous vous surprenez à écrire « import javafx » dans Grille, arrêtez-vous : la question appartient à la vue.

Comment travailler

- Ouvrez le projet dans IntelliJ comme un projet Maven : JavaFX et JUnit sont déclarés dans le pom.xml, rien à installer.
- Comme dans les fascicules, les méthodes à écrire lèvent UnsupportedOperationException ; le message dit l'étape qui les attend. Écrivez-les dans l'ordre des étapes.
- Lancez GrilleTest après chaque étape. L'énoncé dit quels tests doivent passer au vert : les autres peuvent encore échouer, c'est normal.
- Ne lancez le jeu qu'à la fin : la fenêtre appelle presque toutes les méthodes de Grille dès son ouverture.

Les règles du jeu

Une grille de cases cachées, dont certaines cachent une mine. Découvrir une case minée fait perdre. Découvrir une case saine montre combien de mines la touchent, parmi ses huit voisines. On gagne quand toutes les cases saines sont découvertes.

Une case sans mine voisine ne dit rien d'utile : le jeu découvre alors ses voisines à votre place, et ainsi de suite. C'est ce qui ouvre d'un seul clic de grandes zones vides.

Le joueur peut aussi marquer une case. Le premier marquage pose un drapeau, « il y a une mine ici » ; le second un point d'interrogation, « peut-être » ; le troisième efface la marque. Seul le drapeau protège la case d'un clic malheureux : un doute est une note, pas une décision.



Une partie en cours : les chiffres comptent les mines voisines, les astérisques sont des drapeaux.

Les étapes

D.1 Lire ce qui est fourni

Avant d'écrire, lisez Grille. Une partie est déjà là, et vous vous en servirez à chaque étape.

Élément fourni	Ce qu'il fait
`enum Etat`	CACHEE, DECOUVERTE, DRAPEAU, DOUTE : l'état d'une case
`mines, etats`	deux tableaux à deux dimensions, ligne puis colonne
`Grille(lignes, colonnes, mines)`	le constructeur du jeu : des mines au hasard
`Grille(lignes, colonnes)`	le constructeur des tests : une grille vide
`poserMine`	pose une mine où l'on veut, et tient le compte des cases saines
`dansLaGrille`	dit si une position existe : vous en aurez besoin
`etatDe, estMinee, estPerdue`	lisent l'état d'une case ou de la partie

Passe au vert : `grilleImpossible`.

Questions à se poser

- Pourquoi deux constructeurs ? Que deviendraient les tests si les mines étaient toujours placées au hasard ?
- Le constructeur refuse une grille de neuf mines sur neuf cases. Que se passerait-il sans ce contrôle ? Lisez placerAuHasard avant de répondre.

D.2 Compter les mines voisines

Écrivez `minesVoisines(ligne, colonne)` : le nombre de mines parmi les huit cases qui entourent celle-ci. Une case au bord ou dans un coin a moins de huit voisines, et une mine ne se compte pas elle-même.

Indice. Deux boucles imbriquées, de -1 à 1, donnent les neuf décalages autour de la case. Il en reste huit une fois qu'on a écarté le décalage nul, et `dansLaGrille` écarte ceux qui sortent de la grille.

Autour de la case (l, c) : les décalages (dl, dc)

`(-1,-1)`	`(-1,+0)`	`(-1,+1)`
`(+0,-1)`	`(l, c)`	`(+0,+1)`
`(+1,-1)`	`(+1,+0)`	`(+1,+1)`

La voisine de décalage (dl, dc) est la case `(ligne + dl, colonne + dc)`

Le décalage (0, 0) est la case elle-même : on la saute.

Deux boucles, de -1 à +1, parcourent les neuf décalages.

Passe au vert : `minesVoisines`.

D.3 Découvrir une case

Écrivez `decouvrir(ligne, colonne)`, sans la propagation pour l'instant. Dans l'ordre :

- une position hors de la grille, ou une partie déjà perdue : on ne fait rien ;
- une case déjà découverte, ou marquée d'un drapeau : on ne fait rien ; une case en doute, elle, se découvre ;
- sinon la case devient DECOUVERTE, et si elle est minée, la partie est perdue.

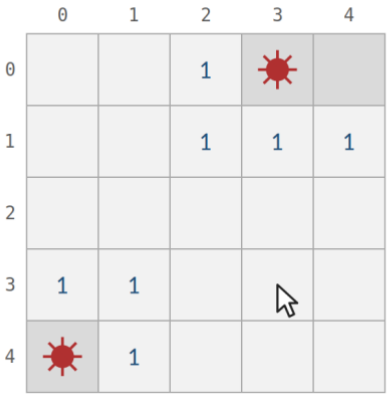
Passent au vert : `propagationLimitee`, `plusRienApresLaFin`.

D.4 La propagation

Complétez `decouvrir` : si la case découverte est saine et n'a aucune mine voisine, découvrez ses huit voisines. Une voisine sans mine voisine découvrira les siennes, et ainsi de suite.

C'est une méthode qui s'appelle elle-même. Elle tient en quelques lignes : la méthode appelle `decouvrir` sur chaque voisine. Ce que vous avez écrit à l'étape D.3 suffit à arrêter la récursion.

Un clic sur la case sous le curseur : deux mines, la zone ouverte en gris clair



La case cliquée n'a aucune mine voisine : ses huit voisines s'ouvrent à leur tour.

Une voisine sans mine voisine propage encore ; une voisine avec un chiffre s'ouvre, puis s'arrête : le chiffre borde la zone.

Les cases encore grises n'ont jamais été visitées : la propagation ne traverse pas un chiffre.

Passe au vert : `propagation`.

Questions à se poser

- Pourquoi la récursion s'arrête-t-elle ? Quelle ligne de l'étape D.3 l'empêche de tourner en rond entre deux cases vides voisines ?
- La propagation découvre-t-elle une case marquée d'un drapeau ? Une case en doute ? Est-ce le comportement que vous attendiez en jouant ?

D.5 Marquer une case

Écrivez `basculerDrapeau(ligne, colonne)` : chaque appel fait passer la case à la marque suivante. Une case découverte ne change plus.

```
CACHEE  ->  DRAPEAU  ->  DOUTE  ->  CACHEE
```

Un `switch` sur l'état de la case convient bien : chaque état dit vers quoi il passe. C'est Grille qui décide de cet enchaînement ; la vue se contente de demander la marque suivante et d'afficher ce qu'on lui rend.

Passent au vert : `marquageEnTroisTemps`, `seulLeDrapeauProtege`, `pasDeMarqueSurUneCaseDecouverte`.

D.6 Gagner, perdre, terminer

Écrivez `estGagnee()` : la partie est gagnée quand toutes les cases saines sont découvertes, et jamais si elle est perdue. Puis `estTerminee()` : perdue ou gagnée.

Indice. Le champ `casesSaines` est tenu à jour par `poserMine` : il dit combien de cases il faut découvrir pour gagner. Il ne reste qu'à compter celles qui le sont.

Passent au vert : `grilleNeuve`, `clicSurMine`, `victoire`.

Les onze tests doivent maintenant passer.

D.7 Jouer

Lancez `LanceurDemineur`, et non `Demineur`. Lancée directement, `Demineur` refuse de démarrer : elle hérite d'une classe de JavaFX, et le lanceur de Java exige alors que JavaFX soit chargé d'une manière que Maven ne prévoit pas. `LanceurDemineur` n'hérite de rien et se contente de démarrer `Demineur` : le contrôle ne se déclenche pas. Son commentaire en dit plus, et le mémento technique y revient en TEC-FX.4.

Si le lancement résiste malgré tout, passez par la fenêtre Maven d'IntelliJ : `Plugins`, puis `javafx`, puis `javafx:run`.

- un clic découvre une case ; un clic droit, Maj+clic ou Ctrl+clic la marque ;
- un clic sur une zone vide en ouvre toute l'étendue d'un coup ;
- un drapeau protège sa case, un point d'interrogation non ;
- la partie annonce la victoire ou la défaite, et ne réagit plus ensuite.

Remarquez que vous n'avez écrit aucune ligne de la fenêtre, et que les règles étaient justes *avant* de l'ouvrir : les tests l'avaient prouvé.

Pour aller plus loin

Le premier clic. Dans la plupart des démineurs, le premier clic ne tombe jamais sur une mine. Ici, il peut. Que faudrait-il changer pour l'empêcher ? Dans Grille, dans la vue, ou dans les deux ? Pourquoi le constructeur actuel ne le permet-il pas ?

Les drapeaux restants. Le jeu d'origine affiche le nombre de mines moins le nombre de drapeaux posés. Quelle méthode faudrait-il ajouter à Grille, et que devrait faire la vue de sa réponse ?

Une grille plus grande. Essayez une grille de cent lignes sur cent colonnes, presque sans mines. Que devient la récursion de la propagation ? Comment pourrait-on s'en passer ?